# Data Science with Rust

A Hands-On Guide with scry-learn

esoc

# Contents


**I Foundations** **5**

**1 Why Rust for Data Science** **7**
1.1 The Case for Rust in Data Science . . . . . 7
1.1.1 Performance . . . . . 7
1.1.2 Safety . . . . . 8
1.1.3 Reproducibility . . . . . 8
1.1.4 When Python is Still Better . . . . . 8
1.2 Setting Up Your Environment . . . . . 9
1.2.1 Install Rust . . . . . 9
1.2.2 Create a New Project . . . . . 9
1.2.3 Add Dependencies . . . . . 9
1.2.4 Essential Cargo Commands . . . . . 10
1.3 Hello, Data Science . . . . . 10
1.3.1 Loading the Dataset . . . . . 10
1.3.2 Summary Statistics . . . . . 11
1.3.3 Making a Scatter Plot . . . . . 11
1.3.4 The Complete Program . . . . . 12
1.4 What Comes Next . . . . . 13

**2 Working with Data** **15**

**3 Visualization Essentials** **17**

**4 Data Cleaning and Preprocessing** **19**

**II Supervised Learning** **21**

**5 Linear Regression** **23**

**6 Classification Fundamentals** **25**

**7 Decision Trees and Random Forests** **27**

**8 Model Evaluation and Selection** **29**

**9 Pipelines** **31**

**10 Gradient Boosting** **33**

**III Unsupervised Learning and Beyond** **35**

**11 Clustering and Segmentation** **37**

**12 Anomaly Detection** **39**

**13 Text Analysis** **41**

**14 Neural Networks** **43**

**15 SVMs and Nearest Neighbors** **45**

**IV Production and Practice** **47**

**16 Hyperparameter Tuning** **49**

**17 Model Explainability** **51**

**18 Probability Calibration** **53**

**19 Ensemble Strategies** **55**

**20 End-to-End Case Study** **57**

**A Rust Crash Course for Data Scientists** **59**

**B scry-learn vs scikit-learn API Map** **61**

**C Performance and Benchmarks** **63**

**D esoc-chart Reference** **65**

# Part I

# Foundations

# Chapter 1

# Why Rust for Data Science

Most data scientists reach for Python. It is the default. The ecosystem is vast, the learning curve is gentle, and nearly every ML paper ships a Python reference implementation. So why would anyone choose Rust?

This chapter answers that question. We will set up a Rust environment, load a real dataset, compute summary statistics, and produce a scatter plot — all before the end of the chapter. By the time you turn the page, you will have a working project and a clear sense of what Rust brings to the table.

## 1.1 The Case for Rust in Data Science

Rust is a systems language. It compiles to native code, manages memory without a garbage collector, and enforces thread safety at compile time. These properties matter for data science more than you might expect.

### 1.1.1 Performance

Rust programs run fast. There is no interpreter overhead, no garbage collector pausing in the middle of a training loop, and no boxing of numeric types unless you ask for it. The compiler applies aggressive optimizations: inlining, autovectorization, loop unrolling, dead code elimination. Zero-cost abstractions mean that iterators compile down to the same machine code as hand-written loops.

When your dataset grows from a CSV on your laptop to a hundred million rows on a server, these properties stop being academic. A pipeline that takes four hours in Python may finish in twenty minutes in Rust — without reaching for C extensions or Cython.

SIMD intrinsics are available when you need them, but in practice the autovectorizer handles most numeric loops. The `ndarray` crate provides

$n$-dimensional arrays with BLAS-backed linear algebra, comparable to NumPy but compiled ahead of time.

### 1.1.2 Safety

Rust's type system catches entire categories of bugs at compile time. There are no null pointers — `Option<T>` forces you to handle the absent case. There are no data races — the borrow checker prevents shared mutable state across threads. There are no use-after-free bugs, no buffer overflows, no silent integer truncation (unless you explicitly call `as`).

For data science, this means fewer midnight debugging sessions. If your pipeline compiles, it will not segfault on row 50 million. Type errors in feature engineering surface immediately, not after an hour of training.

### 1.1.3 Reproducibility

Cargo, Rust's build system, pins every dependency version in `Cargo.lock`. A project that builds today will build identically six months from now, on any platform, with the same compiler version. There is no `pip freeze` drift, no conda environment corruption, no "works on my machine."

Cross-compilation is straightforward. A model trained on Linux can be deployed as a single static binary on a minimal container image — no Python runtime, no shared libraries, no virtualenv.

### 1.1.4 When Python is Still Better

Rust is not always the right tool. Python excels at:

- **Rapid prototyping.** A Jupyter notebook lets you explore data interactively. Rust's compile step, even if fast, adds friction to exploratory work.
- **Ecosystem breadth.** Python has libraries for everything: geospatial analysis, web scraping, bioinformatics, symbolic math. Rust's ecosystem is younger and narrower.
- **Team familiarity.** If your team knows Python and ships products in Python, switching languages has a real cost.

The pragmatic approach: prototype in Python, then rewrite the hot path in Rust. Or, if you are starting fresh and performance matters from day one, start in Rust. This book assumes the latter.

## 1.2 Setting Up Your Environment

### 1.2.1 Install Rust

Install Rust through `rustup`, the official toolchain manager. On Linux or macOS:

```
curl --proto '=https' --tlsv1.2 -sSf
   https://sh.rustup.rs | sh
```

On Windows, download and run `rustup-init.exe` from https://rustup.rs. Accept the defaults. After installation, open a new terminal and verify:

```
rustc --version
cargo --version
```

You should see version 1.75 or later. Any recent stable release will work.

### 1.2.2 Create a New Project

Cargo creates a new project with a single command:

```
cargo new hello_ds
cd hello_ds
```

This generates the following structure:

```
hello_ds/
  Cargo.toml
  src/
    main.rs
```

`Cargo.toml` is the project manifest. `src/main.rs` contains a `main` function that prints "Hello, world!" — we will replace it shortly.

### 1.2.3 Add Dependencies

Open `Cargo.toml` and add two dependencies:

```
[dependencies]
scry-learn = { version = "0.7", features = ["csv"] }
esoc-chart = "0.1"
```

`scry-learn` is the machine learning library we will use throughout this book. The `csv` feature enables loading datasets from CSV files. `esoc-chart` provides visualization through a grammar-of-graphics API.

### 1.2.4 Essential Cargo Commands

Four commands cover most of your workflow:

```
cargo build          # compile the project
cargo run            # compile and run
cargo test           # run tests
cargo doc --open     # generate and open API
    documentation
```

Cargo fetches dependencies automatically on the first build. Subsequent builds are incremental and fast.

## 1.3 Hello, Data Science

Let us do something real. We will load the Iris dataset, print summary statistics, and produce a scatter plot. This is the "Hello, World" of data science, and it will exercise the core workflow: load data, inspect it, visualize it.

### 1.3.1 Loading the Dataset

The Iris dataset contains 150 measurements of three species of iris flowers. Each row has four features (sepal length, sepal width, petal length, petal width) and a class label (setosa, versicolor, or virginica).

`scry-learn` provides a `Dataset` struct that holds feature matrices and target vectors together. The `from_csv` constructor reads a CSV file and designates one column as the target:

```rust
use scry_learn::dataset::Dataset;

let ds =
    Dataset::from_csv("datasets/iris/iris.csv",
    "species")
     .expect("failed to load iris.csv");
```

The second argument, `"species"`, names the target column. All remaining columns become features. Categorical targets are automatically encoded as integers, and the original class labels are stored in `ds.class_labels`.

### 1.3.2 Summary Statistics

Call `describe()` to print a summary table:

```
ds.describe();
```

This prints a table like the following:

```
                 count    mean     std      min      25%
                         50%      75%      max
sepal_length   150      5.843    0.828    4.300    5.100
      5.800    6.400    7.900
sepal_width    150      3.054    0.434    2.000    2.800
      3.000    3.300    4.400
petal_length   150      3.759    1.764    1.000    1.600
      4.350    5.100    6.900
petal_width    150      1.199    0.763    0.100    0.300
      1.300    1.800    2.500
```

You immediately see the scale of each feature. Sepal length ranges from 4.3 to 7.9; petal width from 0.1 to 2.5. This kind of inspection is the first step in any analysis.

### 1.3.3 Making a Scatter Plot

The `esoc-chart` crate provides an express API for common plots. To make a scatter plot of sepal length versus sepal width, colored by species, we need the feature columns and the class labels:

```
use esoc_chart::express::scatter;

// Extract feature columns
let sepal_length = ds.column("sepal_length");
let sepal_width = ds.column("sepal_width");

// Map encoded target back to species names
let species: Vec<String> = ds.target.iter()
    .map(|&t| {
        ds.class_labels.as_ref().unwrap()[t as
            usize].clone()
    })
    .collect();

// Build the scatter plot
let svg = scatter(&sepal_length, &sepal_width)
    .color_by(&species)
    .title("Iris: Sepal Length vs Width")
```

```
    .x_label("Sepal Length")
    .y_label("Sepal Width")
    .to_svg();

std::fs::write("iris_scatter.svg", &svg)
    .expect("failed to write SVG");

println!("Wrote iris_scatter.svg");
```

This produces an SVG file with three color-coded clusters. Setosa separates cleanly from the other two species along sepal width. Versicolor and virginica overlap — a pattern we will revisit when we build classifiers in Chapter 6.

### 1.3.4 The Complete Program

Here is the full `src/main.rs`:

```
use scry_learn::dataset::Dataset;
use esoc_chart::express::scatter;

fn main() {
    // Load the Iris dataset
    let ds = Dataset::from_csv(
        "datasets/iris/iris.csv", "species"
    ).expect("failed to load iris.csv");

    // Print summary statistics
    ds.describe();

    // Extract feature columns
    let sepal_length = ds.column("sepal_length");
    let sepal_width = ds.column("sepal_width");

    // Map encoded target back to species names
    let species: Vec<String> = ds.target.iter()
        .map(|&t| {
            ds.class_labels
                .as_ref()
                .unwrap()[t as usize]
                .clone()
        })
        .collect();

    // Build a scatter plot
    let svg = scatter(&sepal_length, &sepal_width)
        .color_by(&species)
```

```
        .title("Iris: Sepal Length vs Width")
        .x_label("Sepal Length")
        .y_label("Sepal Width")
        .to_svg();

    std::fs::write("iris_scatter.svg", &svg)
        .expect("failed to write SVG");

    println!("Wrote iris_scatter.svg");
}
```

Listing 1.1: src/main.rs — Hello Data Science

Run it:

```
cargo run
```

Cargo compiles the project, fetches dependencies, and executes the binary. You should see the summary statistics table followed by the confirmation message. Open `iris_scatter.svg` in a browser to see the plot.

## 1.4 What Comes Next

This book is organized in four parts.

**Part I: Foundations** (Chapters 1–4) covers the prerequisites: working with tabular data, visualization, and data cleaning. These chapters establish the patterns you will use in every subsequent chapter.

**Part II: Supervised Learning** (Chapters 5–10) builds models that predict outcomes from labeled data. We start with linear regression, move through decision trees and random forests, and finish with gradient boosting. Each chapter includes model evaluation and practical advice on when to use which algorithm.

**Part III: Unsupervised Learning and Beyond** (Chapters 11–15) covers clustering, anomaly detection, text analysis, neural networks, and support vector machines. These methods find structure in data without explicit labels, or operate in domains where the supervised framing is less natural.

**Part IV: Production and Practice** (Chapters 16–20) addresses the concerns that arise when models leave the notebook: hyperparameter tuning, explainability, calibration, ensembling, and a complete end-to-end case study that ties everything together.

Turn the page. We have data to work with.

# Chapter 2

# Working with Data

# Chapter 3

# Visualization Essentials

# Chapter 4

# Data Cleaning and Preprocessing

# Part II

# Supervised Learning

# Chapter 5

# Linear Regression

# Chapter 6

# Classification Fundamentals

# Chapter 7

# Decision Trees and Random Forests

# Chapter 8

# Model Evaluation and Selection

# Chapter 9

# Pipelines

# Chapter 10

# Gradient Boosting

# Part III

# Unsupervised Learning and Beyond

# Chapter 11

# Clustering and Segmentation

# Chapter 12

# Anomaly Detection

# Chapter 13

# Text Analysis

# Chapter 14

# Neural Networks

# Chapter 15

# SVMs and Nearest Neighbors

# Part IV

# Production and Practice

# Chapter 16

# Hyperparameter Tuning

# Chapter 17

# Model Explainability

# Chapter 18

# Probability Calibration

# Chapter 19

# Ensemble Strategies

# Chapter 20

# End-to-End Case Study

# Appendix A

# Rust Crash Course for Data Scientists

# Appendix B

# scry-learn vs scikit-learn API Map

# Appendix C

# Performance and Benchmarks

# Appendix D

# esoc-chart Reference